NTA Data Analysis

Project Blog

CSC1049 – Year 3 Project

Kevin Biju – 22332903

Daragh Montague – 22375933

Blog 1 – Project Kickoff

October 2025

Picking the Project

We spent the first few weeks figuring out what we actually wanted to build. We both use public transport to get to DCU and it's pretty frustrating when buses just don't show up or are 15 minutes late with no warning. The apps that exist right now (TFI Live, Google Maps) tell you when the next bus is "supposed" to arrive but they don't really tell you if it's going to be on time or not.

We found out that the NTA (National Transport Authority) has a public API that gives you real-time vehicle updates in a format called GTFS-Realtime. There's also static GTFS data with all the timetables. The idea was: if we poll the real-time feed over time, we can build up a picture of which routes are actually reliable and which ones aren't.

Early Decisions

We had a few calls early on to figure out the tech stack. Daragh had done some work with Python data pipelines before so that was a natural fit for the backend. I wanted to do the frontend in React since I'd used it in other modules. For the database we went with PostgreSQL because we'd need to do a lot of joins between the static timetable data and the real-time updates, and Postgres handles that well.

For machine learning we decided on XGBoost through scikit-learn. We looked at a few options but XGBoost seemed like the best fit for tabular regression data, which is basically what delay prediction is.

Proposal

We submitted the project proposal in mid-October. The core pitch was: a web app that shows you how reliable different bus, rail, and tram routes are based on real data, not just timetables. We also wanted a journey planner that ranks results by reliability instead of just travel time.

What's Next

The functional spec is due in November so we need to nail down exactly what features we're promising. We've started sketching out the database schema and thinking about how the data pipeline will work.

Blog 2 – Functional Spec and Design

November 2025

Functional Spec

We submitted the functional spec in early November. Writing it was useful because it forced us to actually think through what the app needs to do rather than just having a vague idea. We broke it into functional requirements (what it does) and non-functional requirements (how well it does it).

The main features we committed to:

- Real-time data collection from the NTA API
- A delay prediction model using machine learning
- An interactive dashboard with reliability scores
- A journey planner that considers reliability
- Filtering by route, day, and time

We also promised weather data integration and comparison with Google Maps travel times, which might be ambitious but we'll see how it goes.

System Design

Around the same time we put together two design documents. One covers the overall pipeline architecture and the other covers the initial database schema.

The pipeline design is fairly straightforward: poll the NTA real-time feed, store the raw data, join it with the static timetables to figure out how late each arrival was, then feed that into the ML model. The tricky part is making sure it all runs continuously without falling over.

For the database, we went with separate schemas in PostgreSQL: one for static GTFS data (routes, stops, trips, timetables), one for the raw real-time snapshots, and one for our derived analytics. Keeping them separate makes it cleaner and means we can reload static data without touching the real-time stuff.

Splitting the Work

We decided on a clear split: Daragh handles the data side (pipelines, ingestion, ML model) and I handle the user-facing side (API, dashboard, visualisation). The database and integration work is shared. This way we can work independently most of the time and just sync up on calls every couple of days.

Initial Code Structure

We set up the project repo on GitLab and created the basic directory structure. The code lives under a code/ directory with subdirectories for the API, ingestion pipelines, ML, database migrations, and the dashboard. Nothing actually runs yet but having the structure in place makes it easier to work in parallel.

Blog 3 – Dev Environment and Database

Late December 2025 – January 2026

Christmas Break Progress

We used the Christmas break to get the actual development environment working. Before this we just had the project structure and documents, no running code.

The first thing was getting PostgreSQL running. We went with Docker so we wouldn't have to install Postgres directly on our machines. The docker-compose file spins up a Postgres 16 container on port 5433. We hit a few issues with this early on because Daragh is on a different setup to me and we had to make sure the config worked on both.

Database Migrations

In January we built out the database schema using Alembic for migrations. This was actually more work than we expected. The static GTFS spec has quite a lot of tables – agency, routes, stops, calendar, calendar_dates, trips, stop_times, and shapes. They all have foreign key relationships between them and we needed indexes on the right columns to keep queries fast.

We had a painful bug where one migration was trying to add columns that already existed from an earlier migration. It took a while to track down because the error messages from Alembic aren't always obvious. We ended up having to make that migration a no-op since the columns were already there.

There's also a staging schema (gtfs_static_staging) that we use for loading new static data. The idea is you load everything into staging first, check it looks right, then swap it into the production schema. That way if a download is corrupted or half-finished it doesn't mess up the live data.

Connecting the Pieces

By the end of January we had Docker running, the database schema set up with all the GTFS tables, and Alembic tracking our migrations. Not the most exciting milestone but without this foundation nothing else would work.

Blog 4 – Building the Data Pipeline

Early February 2026

Static GTFS Pipeline

Daragh built the static GTFS ingestion pipeline. It downloads the ZIP file from the NTA, validates it has all the required files (routes.txt, stops.txt, trips.txt, etc.), transforms the data into the right formats, and loads it into PostgreSQL. There's a lot of data – over 14,000 stops, 800+ routes, and 200,000+ trips.

One annoying issue we ran into was file encoding. On Windows, some files were getting saved as UTF-16 instead of UTF-8, which caused weird errors when trying to load them. We had to convert them before importing. This happened with both the requirements.txt and some database dump files. Small thing but it wasted a good hour or two of debugging.

Real-Time Polling

The real-time pipeline polls the NTA GTFS-Realtime TripUpdates endpoint. The NTA API requires authentication and we had to figure out the right way to pass the API key. The poller runs on a loop (every 30 seconds by default) and stores each response as a raw snapshot in the database, including the protobuf payload, HTTP metadata, and a SHA256 hash.

We use ETags for conditional requests so if the data hasn't changed since the last poll we don't re-download it. This saves bandwidth and avoids storing duplicate snapshots.

Delay Event Construction

This is where the interesting stuff happens. Each raw snapshot gets parsed and joined to the static timetable data. For every stop in every trip update, we match the predicted arrival time to the scheduled arrival time and calculate the delay. These get stored as delay events in the analytics schema.

Getting the join right was tricky. The real-time data references trips and stops by ID, and we need to find the matching scheduled time from the stop_times table. We also had an issue where data imported from Daragh's machine had different snapshot IDs to the ones generated on mine, which broke the sequential tracking. We switched to using timestamps instead of IDs which fixed it.

ML Model

Daragh got the XGBoost model working. It trains on the last 7 days of delay events using features like route, stop sequence, time of day, and day of week. The model predicts delay in seconds. We log MAE and other metrics each time it trains so we can track how it's doing over time.

Pipeline Orchestrator

Everything ties together in run_pipeline.py which manages the whole cycle: continuous RT polling, analytics rebuilds every 5 minutes, and model retraining every 3 hours. It runs as a single long-lived process.

Blog 5 – Dashboard and API

Mid February 2026

Backend API

With the data pipeline running and delay events building up in the database, I started on the backend API. We went with FastAPI because it's fast to develop with and gives you automatic Swagger docs which are handy for testing.

The API is split into five modules: routes, stops, trips, search, and analytics. The routes and stops endpoints are fairly standard CRUD stuff. The interesting ones are search and analytics.

The search endpoint is the core of the app. For direct routes, it finds stops near your origin and destination using the Haversine formula in SQL, then checks which routes serve both areas. For transfers, it finds hub stops where two routes overlap and builds two-leg itineraries. Results get ranked by reliability first, not just distance.

The analytics endpoints return on-time percentage, average delay, and sample size for any route or stop. You can filter by day of week and hour which is useful for seeing patterns like "this route is fine on weekends but awful at 8am on Mondays."

Frontend Dashboard

I set up the dashboard with React, TypeScript, and Vite. For styling I used Tailwind CSS with a glassmorphic design – frosted glass cards with backdrop blur. It looks clean and modern.

The hardest part was the journey search page. The autocomplete inputs need to query stops by name with debouncing so it doesn't fire a request on every keystroke. Users can also click the map to set a location, which required managing state for which field is currently in "map selection" mode.

The home page ended up with quite a lot on it: a hero section, favourites, a Google Map showing stops, stats cards, a pipeline status indicator, and a list of transport operators grouped by type.

Google Maps

We integrated the Google Maps JavaScript API for all the maps in the app. It supports dark mode with a custom style, different marker colours for origin/destination/stops, info windows, and polyline route rendering. Getting the API key set up was straightforward but we have to be careful not to commit it to git.

Bringing It Together

The best moment was when we got the full loop working for the first time: the pipeline polling live data, analytics building up, and the dashboard showing real reliability scores. It went from being a bunch of separate pieces to an actual working app. We had a call where we both tested it and found a bunch of small bugs but the core flow worked.

Blog 6 – Final Push and Reflections

Late February 2026

Last Two Weeks

The last couple of weeks were busy. We added transfer journey support so the search can find routes that require changing at a hub stop, which made the journey planner much more useful. Before that it could only find direct routes, which limits you quite a lot in Dublin where you often need to take two buses.

We also built the trip summary page that gives a full breakdown of a selected journey – each leg with its own reliability stats, walking distances, and a map showing origin, destination, and transfer points. For transfer journeys it calculates a combined reliability by multiplying the individual leg reliabilities together.

On the route details page we added day and hour filters so you can check how a specific route does at different times. This is actually one of the most useful features because reliability varies a lot throughout the day.

Documentation

We wrote up the technical specification and user manual. The tech spec covers the full architecture: database schema, pipeline design, API endpoints, ML system, and the frontend. The user manual is more of a walkthrough of how to actually use the app – how to search for a journey, what the reliability gauges mean, how favourites work, that kind of thing.

Challenges Along the Way

Looking back, the main challenges were:

- Encoding issues on Windows – files kept getting saved as UTF-16 instead of UTF-8 which caused all sorts of import errors. Small but annoying.
- Database migration conflicts – Alembic migrations clashing when we both made schema changes independently. We had to merge branches and create no-op migrations to fix it.
- Data mismatch between environments – Daragh's imported data had different ID sequences to mine, so anything relying on sequential IDs broke. Switching to timestamp-based tracking fixed it.
- Tailwind version headaches – Tailwind CSS v4 came out and broke everything. Had to make sure we stayed on v3.4.x.
- Zombie processes – Node and Python processes would pile up on our machines if we didn't kill them properly before restarting. We got into the habit of checking for stale processes.

What We'd Do Differently

If we were starting again, we'd probably set up CI/CD earlier. Right now testing is manual and it's easy to push something that breaks. We'd also start collecting real-time data earlier in the

project – the more data you have the better the reliability scores are, and we only started the pipeline in February.

The ML model accuracy isn't as high as we'd like. The delay prediction is useful as a general indicator but it's not precise enough to give exact minute-by-minute predictions. With more data and possibly more features (weather, time of year, special events) it could improve.

What We're Happy With

The journey planner works well and the reliability scores are genuinely useful. You can see patterns in the data that you'd only know from experience otherwise – like which routes are consistently late during rush hour. The glassmorphic design looks professional and the dark mode is nice. The pipeline has been running continuously without issues.

Overall we're happy with how it turned out. It does what we set out to do: give people a way to check how reliable their bus or train actually is, not just what the timetable says.